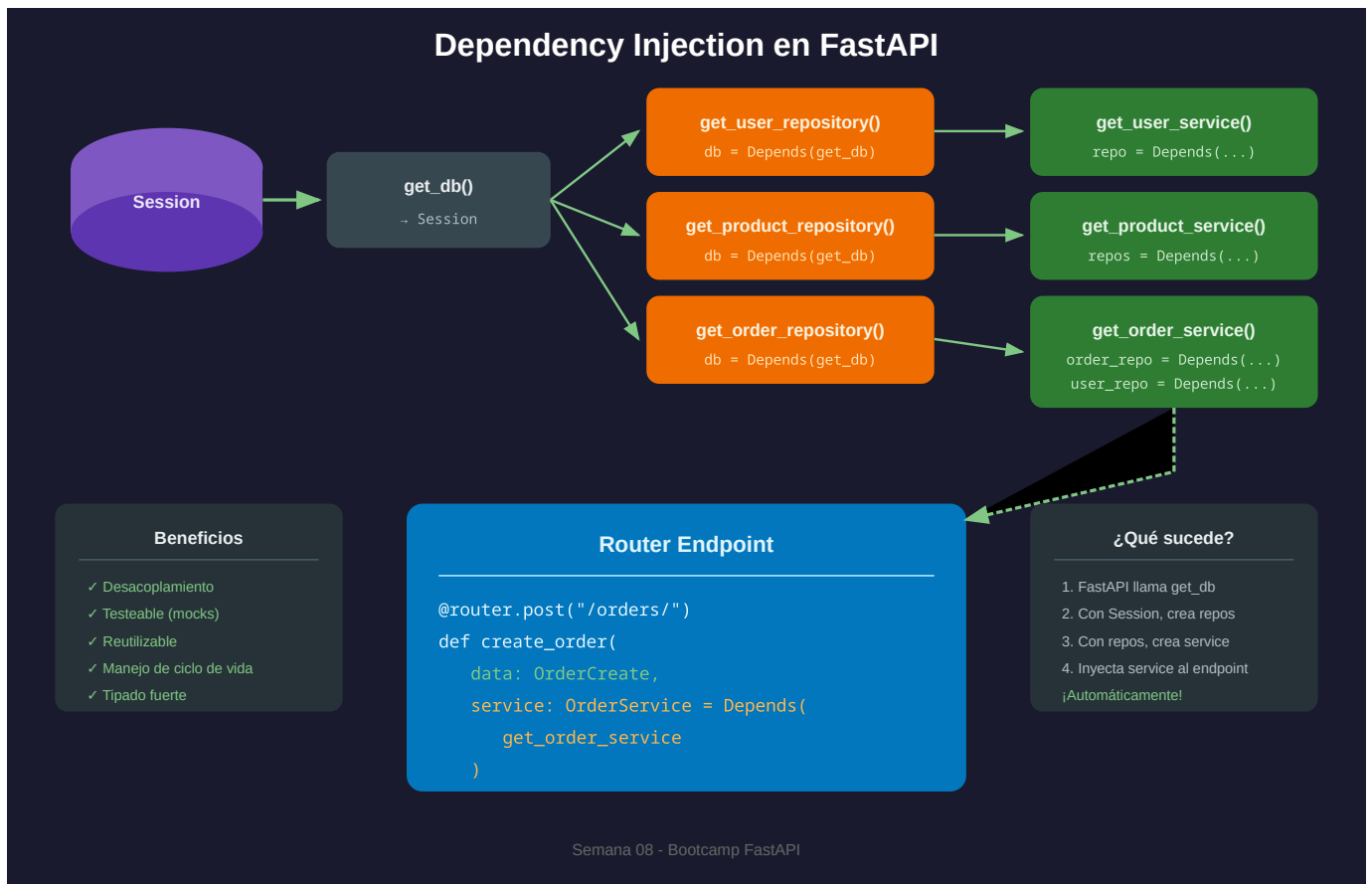


Patrones Factory y Builder



Objetivos

- Entender el patrón Factory y cuándo usarlo
- Implementar el patrón Builder para objetos complejos
- Aplicar estos patrones en FastAPI
- Simplificar la creación de objetos en tests

Patrón Factory

El patrón **Factory** encapsula la lógica de creación de objetos, centralizando y simplificando la instanciación.

Problema

```
# ❌ Creación repetida y dispersa
# En router
product = Product(
    name=data.name,
    sku=data.sku,
    price=data.price,
    stock=data.stock,
    created_at=datetime.utcnow(),
    is_active=True
)
```

```

# En test
product = Product(
    name="Test Product",
    sku="TST-0001",
    price=99.99,
    stock=10,
    created_at=datetime.utcnow(),
    is_active=True
)

# En seed/fixture
product = Product(
    name="Seed Product",
    sku="SEED-001",
    price=50.0,
    stock=100,
    created_at=datetime.utcnow(),
    is_active=True
)

```

Solución: Factory

```

# factories/product.py
from datetime import datetime
from models.product import Product
from schemas.product import ProductCreate

class ProductFactory:
    """Factory para crear instancias de Product"""

    @staticmethod
    def create(data: ProductCreate) -> Product:
        """Crea Product desde DTO"""
        return Product(
            name=data.name,
            sku=data.sku,
            price=data.price,
            stock=data.stock,
            created_at=datetime.utcnow(),
            is_active=True
        )

    @staticmethod
    def create_for_test(
        name: str = "Test Product",
        sku: str | None = None,
        price: float = 99.99,
        stock: int = 10
    ) -> Product:
        """Crea Product para testing con valores por defecto"""
        import uuid
        return Product(
            name=name,
            sku=sku or f"TST-{uuid.uuid4().hex[:4].upper()}",

```

```

        price=price,
        stock=stock,
        created_at=datetime.utcnow(),
        is_active=True
    )

```

Uso

```

# En Service
class ProductService:
    def create(self, data: ProductCreate) -> Product:
        product = ProductFactory.create(data)
        return self.repo.add(product)

# En Test
def test_product_service():
    product = ProductFactory.create_for_test(name="Widget", price=25.0)
    # ...

```

Patrón Builder

El patrón **Builder** construye objetos complejos paso a paso, especialmente útil cuando hay muchos parámetros opcionales.

Problema

```

# ❌ Constructor con muchos parámetros
order = Order(
    user_id=user.id,
    status=OrderStatus.PENDING,
    total=0,
    discount=0,
    tax=0,
    shipping_cost=0,
    shipping_address="123 Main St",
    billing_address="123 Main St",
    notes="Gift wrap please",
    created_at=datetime.utcnow(),
    items=[]
)

```

Solución: Builder

```

# builders/order.py
from datetime import datetime
from models.order import Order, OrderItem, OrderStatus

class OrderBuilder:
    """Builder para construir Order paso a paso"""

    def __init__(self):
        self._user_id: int | None = None

```

```

self._items: list[OrderItem] = []
self._discount: float = 0
self._shipping_address: str | None = None
self._billing_address: str | None = None
self._notes: str | None = None

def for_user(self, user_id: int) -> "OrderBuilder":
    """Establece el usuario"""
    self._user_id = user_id
    return self

def add_item(
    self,
    product_id: int,
    quantity: int,
    price: float
) -> "OrderBuilder":
    """Agrega un item al pedido"""
    item = OrderItem(
        product_id=product_id,
        quantity=quantity,
        unit_price=price,
        subtotal=quantity * price
    )
    self._items.append(item)
    return self

def with_discount(self, discount: float) -> "OrderBuilder":
    """Aplica descuento"""
    self._discount = discount
    return self

def ship_to(self, address: str) -> "OrderBuilder":
    """Establece dirección de envío"""
    self._shipping_address = address
    return self

def bill_to(self, address: str) -> "OrderBuilder":
    """Establece dirección de facturación"""
    self._billing_address = address
    return self

def with_notes(self, notes: str) -> "OrderBuilder":
    """Agrega notas"""
    self._notes = notes
    return self

def build(self) -> Order:
    """Construye el Order final"""
    if not self._user_id:
        raise ValueError("User ID is required")
    if not self._items:
        raise ValueError("At least one item is required")
    if not self._shipping_address:
        raise ValueError("Shipping address is required")

```

```

# Calcular totales
subtotal = sum(item.subtotal for item in self._items)
tax = subtotal * 0.16 # 16% IVA
shipping = 10.0 if subtotal < 100 else 0 # Envío gratis > $100
total = subtotal - self._discount + tax + shipping

return Order(
    user_id=self._user_id,
    status=OrderStatus.PENDING,
    subtotal=subtotal,
    discount=self._discount,
    tax=tax,
    shipping_cost=shipping,
    total=total,
    shipping_address=self._shipping_address,
    billing_address=self._billing_address or self._shipping_address,
    notes=self._notes,
    created_at=datetime.utcnow(),
    items=self._items
)

```

Uso

```

# Fluent interface - fácil de leer
order = (
    OrderBuilder()
    .for_user(user.id)
    .add_item(product_id=1, quantity=2, price=29.99)
    .add_item(product_id=3, quantity=1, price=49.99)
    .with_discount(10.0)
    .ship_to("123 Main St, City")
    .with_notes("Gift wrap please")
    .build()
)

```

Factory para Dependencies

```

# factories/dependencies.py
from sqlalchemy.orm import Session

from repositories.product import ProductRepository
from repositories.order import OrderRepository
from services.product import ProductService
from services.order import OrderService
from unit_of_work import UnitOfWork

class ServiceFactory:
    """Factory para crear services con sus dependencias"""

    @staticmethod
    def create_product_service(db: Session) -> ProductService:
        repo = ProductRepository(db)

```

```

        return ProductService(repo)

    @staticmethod
    def create_order_service(db: Session) -> OrderService:
        uow = UnitOfWork(db)
        return OrderService(uow)

# dependencies.py
from fastapi import Depends
from database import get_db

def get_product_service(db: Session = Depends(get_db)) -> ProductService:
    return ServiceFactory.create_product_service(db)

def get_order_service(db: Session = Depends(get_db)) -> OrderService:
    return ServiceFactory.create_order_service(db)

```

Factories para Testing

```

# tests/factories.py
from models.user import User
from models.product import Product
from models.order import Order

class TestDataFactory:
    """Factory para crear datos de prueba"""

    _user_counter = 0
    _product_counter = 0

    @classmethod
    def create_user(
        cls,
        username: str | None = None,
        email: str | None = None,
        is_active: bool = True
    ) -> User:
        cls._user_counter += 1
        return User(
            username=username or f"user_{cls._user_counter}",
            email=email or f"user_{cls._user_counter}@test.com",
            password_hash="hashed_password",
            is_active=is_active
        )

    @classmethod
    def create_product(
        cls,
        name: str | None = None,
        price: float = 99.99,

```

```

        stock: int = 10
    ) -> Product:
        cls._product_counter += 1
        return Product(
            name=name or f"Product {cls._product_counter}",
            sku=f"PRD-{cls._product_counter:04d}",
            price=price,
            stock=stock
        )

    @classmethod
    def reset_counters(cls):
        """Reset para entre tests"""
        cls._user_counter = 0
        cls._product_counter = 0

# Uso en tests
def test_order_creation():
    user = TestDataFactory.create_user()
    product1 = TestDataFactory.create_product(price=25.0)
    product2 = TestDataFactory.create_product(price=50.0)

    order = (
        OrderBuilder()
        .for_user(user.id)
        .add_item(product1.id, 2, product1.price)
        .add_item(product2.id, 1, product2.price)
        .ship_to("Test Address")
        .build()
    )

    assert order.total > 0

```



Cuándo Usar Cada Patrón

Patrón	Usar cuando...
Factory	Creación simple con variantes (test, prod)
Builder	Muchos parámetros opcionales
Factory	Encapsular lógica de instanciación
Builder	Construcción paso a paso con validación



Checklist

- ☐ Sé crear Factories para entidades
- ☐ Puedo implementar Builder para objetos complejos
- ☐ Entiendo cuándo usar cada patrón
- ☐ Sé crear factories para testing